

Resource Booking API

Complete Postman endpoint testing guide

Before you test

Run the application from **C:\Users\HPI\Documents\Booking\Booking** using **mvn spring-boot:run**. The default server address is **http://localhost:8080**. MySQL must be running and configured through **DB_URL**, **DB_USERNAME**, and **DB_PASSWORD** if the defaults do not match your system.

Create a Postman environment

Variable	Initial value
baseUrl	http://localhost:8080
adminToken	Leave blank; fill after admin login
userToken	Leave blank; fill after user login
resourceId	Leave blank; fill after resource creation
reservationId	Leave blank; fill after reservation creation

For every JSON request use Header: **Content-Type: application/json**. For protected endpoints use Authorization type **Bearer Token** and paste the appropriate token.

1. Authentication

POST **{{baseUrl}}/auth/login** Public

Log in and receive a JWT. Do not escape the @ character in the password.

Request body:

```
{
  "username": "admin",
  "password": "Admin@123"
}
```

Expected: 200 OK. Copy the token value to adminToken.

POST **{{baseUrl}}/auth/login** Public

Log in as a normal user.

Request body:

```
{
  "username": "user",
  "password": "User@123"
}
```

Expected: 200 OK. Copy the token value to userToken.

Seed users

ADMIN: admin / Admin@123. USER: user / User@123. If an older database already has these usernames, the initializer intentionally does not overwrite their password or role. Confirm the users table stores roles ADMIN and USER.

2. Resource endpoints

First log in as ADMIN and set Authorization to Bearer {{adminToken}} for create, update, and delete.

POST	{{baseUrl}}/resources	ADMIN
-------------	------------------------------	-------

Create a bookable resource. Save the response id as resourceId.

Request body:

```
{
  "name": "Conference Room A",
  "description": "Room with projector and 12 seats",
  "available": true,
  "price": 500.00
}
```

Expected: 201 Created.

GET	{{baseUrl}}/resources	ADMIN or USER
------------	------------------------------	---------------

Retrieve every resource.

Expected: 200 OK.

GET	{{baseUrl}}/resources/{{resourceId}}	ADMIN or USER
------------	---	---------------

Retrieve one resource by Long ID.

Expected: 200 OK; 404 if the ID does not exist.

PUT	{{baseUrl}}/resources/{{resourceId}}	ADMIN
------------	---	-------

Replace the resource values.

Request body:

```
{
  "name": "Conference Room A",
  "description": "Updated room description",
  "available": true,
  "price": 650.00
}
```

Expected: 200 OK.

DELETE	{{baseUrl}}/resources/{{resourceId}}	ADMIN
---------------	---	-------

Delete the resource after completing reservation tests.

Expected: 204 No Content.

3. Reservation endpoints

Create a resource first. Use a resource with available set to true. USER and ADMIN can create a reservation; the authenticated JWT always determines the reservation owner.

POST	{{baseUrl}}/reservations	ADMIN or USER
-------------	---------------------------------	---------------

Create a reservation. Set Bearer {{userToken}} to test USER behavior. Do not send userId; it is not accepted by the request DTO.

Request body:

```
{
  "resourceId": {{resourceId}},
  "startTime": "2026-09-10T10:00:00",
  "endTime": "2026-09-10T12:00:00"
}
```

Expected: 201 Created. Save response id as reservationId. Status is PENDING and price comes from the resource.

GET	{{baseUrl}}/reservations	ADMIN or USER
------------	---------------------------------	---------------

List reservations. ADMIN receives all; USER receives only owned reservations.

Expected: 200 OK with a paginated Page response.

GET	{{baseUrl}}/reservations/{{reservationId}}	ADMIN or owning USER
------------	---	----------------------

Get a single reservation. A USER attempting to read another user's reservation receives 403.

Expected: 200 OK, 403 Forbidden, or 404 Not Found.

PUT	{{baseUrl}}/reservations/{{reservationId}}	ADMIN
------------	---	-------

Update resource/times and optionally status.

Request body:

```
{
  "resourceId": {{resourceId}},
  "startTime": "2026-09-10T10:00:00",
  "endTime": "2026-09-10T12:00:00",
  "status": "CONFIRMED"
}
```

Expected: 200 OK. Status may be PENDING, CONFIRMED, or CANCELLED.

DELETE	{{baseUrl}}/reservations/{{reservationId}}	ADMIN
---------------	---	-------

Delete the reservation.

Expected: 204 No Content.

4. Reservation filter, pagination, and sorting

Test URL	Purpose
{{baseUrl}}/reservations?status=PENDING	Only PENDING reservations.
{{baseUrl}}/reservations?minPrice=100&maxPrice=1000	Reservations within the inclusive price range.
{{baseUrl}}/reservations?page=0&size=10	First page containing at most ten results.
{{baseUrl}}/reservations?sortBy=price&sortDirection=desc	Highest price first.
{{baseUrl}}/reservations?status=PENDING&minPrice=100&maxPrice=1000&page=0&size=10&sortBy=startTime&sortDirection=asc	Full required filtering, pagination, and sorting example.

Use Bearer {{adminToken}} to confirm all matching reservations are returned. Use Bearer {{userToken}} to confirm the exact same filters are restricted to that user's reservations.

5. Required security and validation checks

Test	How	Expected result
No token	GET /resources without Authorization	401 Unauthorized
Bad token	GET /resources with Bearer invalid-token	401 Unauthorized
USER resource mutation	POST /resources using userToken	403 Forbidden
USER reservation mutation	PUT or DELETE /reservations/{id} using userToken	403 Forbidden
Ownership	Create a reservation as one USER; request it using another USER token	403 Forbidden
Missing resource fields	POST /resources with name "" or missing price	400 Bad Request
Negative price	POST /resources with price -10	400 Bad Request
Invalid time range	POST /reservations where endTime is before startTime	400 Bad Request
Invalid status	PUT /reservations/{id} with status "INVALID"	400 Bad Request

Suggested Postman execution order

1. Start MySQL and the Spring Boot application.
2. Log in as ADMIN; save the returned token as adminToken.
3. Create a resource as ADMIN; save its id as resourceId.
4. Log in as USER; save the returned token as userToken.
5. Read resources as USER.
6. Create a reservation as USER; save its id as reservationId.
7. List/filter reservations as USER and verify only that user's results are returned.
8. Retrieve the reservation by ID as USER.
9. Use adminToken to list all reservations and update the USER reservation status.
10. Execute the permission and validation checks on the previous page.
11. Delete the reservation and resource as ADMIN if cleanup is desired.

Swagger UI

When the application is running, interactive OpenAPI documentation is available at <http://localhost:8080/swagger-ui/index.html>.